Plan du cours

- POO :
  - objets, classes, Encapsulation, Héritage, polymorphisme.
- Programmation C++ :
  - spécificités du C++, classes et objets, fonctions membres, constructeur de recopie,
  - fonctions amies, surcharge des opérateurs, héritage simple,
  - héritage multiple, classe abstraite, les flots, la gestion des exceptions.

POO :
Programmation Orientée Objet

Exemple

- Prenons ces informations :
  - 6 Novembre 1975 (Marche Verte)
  - 18 Novembre 1955 (L'indépendance)
  - 07 Mars 2005 (une date)
- Ce sont des dates.
- Chaque date se caractérise par :
  - un *jour*
  - un *mois*
  - une *année*.

Les Dates

Date

- Jour
- Mois
- Année

Marche Verte

- 6
- Novembre
- 1975

Une Date

- 7
- Mars
- 2005

L’indépendance

- 18
- Novembre
- 1955

Classes

- Le *Jour*, le *Mois* et l'*Année* sont les attributs d'une Date.

- Cet ensemble d'attributs est appelé une Classe.

Objets

- Le 6 Novembre 1975 et le 18 Novembre 1955 sont chacune des instances de la classe Date.

- Chacune de ces dates est appelée un Objet.

Encore un Exemple

- Pierre, 19 ans, étudiant en TM. groupe 1, sous-groupe 2.
- Nathalie, étudiante en informatique, a 18 ans. Elle est dans le sous-groupe 1 du Groupe 2.
- Paul, un garçon de 21 ans, est en GODT. Il est dans le groupe 3, sous-groupe 2.

Etudiant

Ici, un étudiant se caractérise par

- age
- département
- groupe
- sous-groupe
- sexe

Etudiant

Etudiant

- Prénom
- Age
- Departement
- Groupe
- SousGroupe
- Sexe

Etudiant3

- Nathalie
- 18
- INFO
- 2
- 1
- Fille

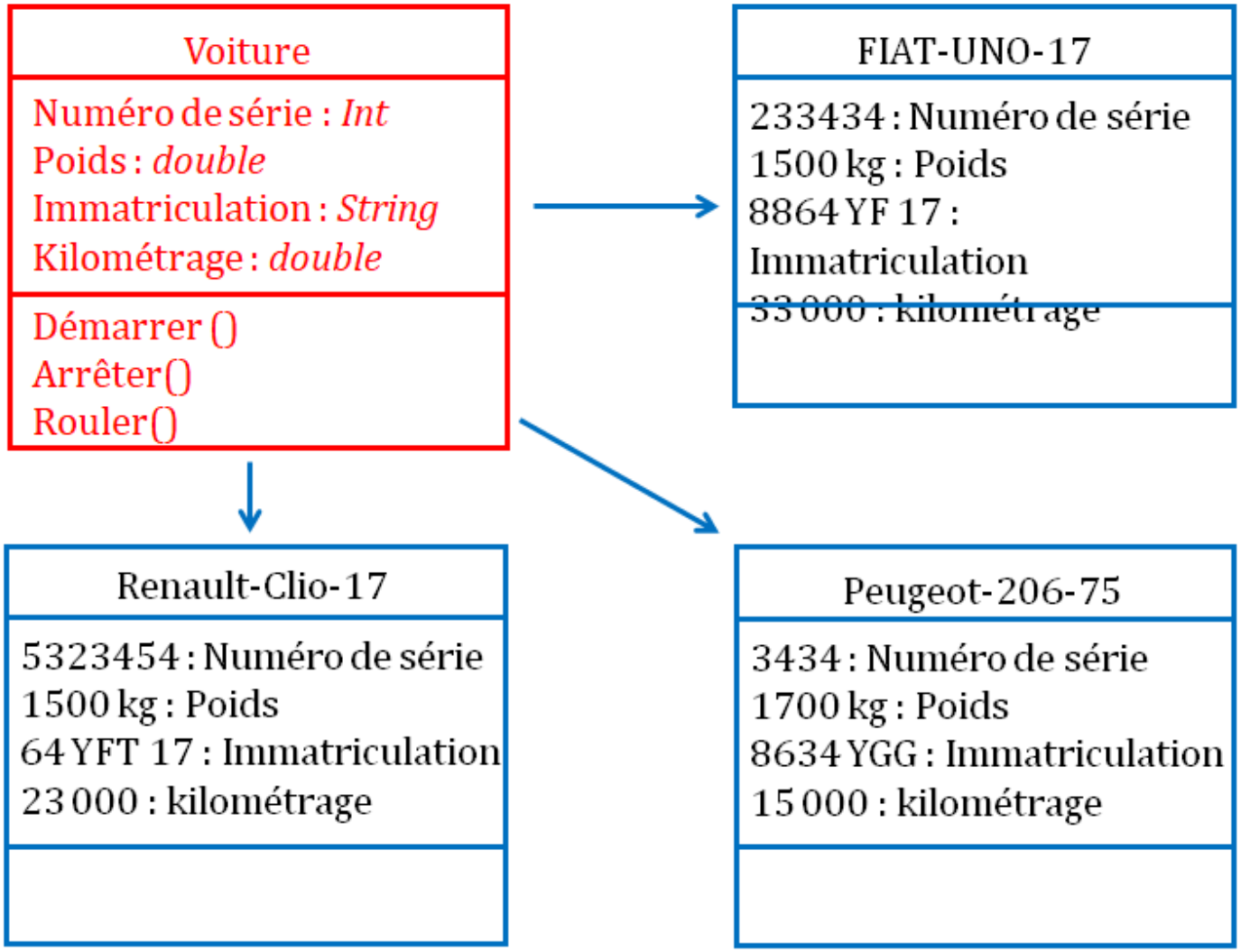
Etudiant1

- Pierre
- 19
- TM
- 1
- 2
- Garçon

Etudiant2

- Paul
- 21
- GODT
- 3
- 2
- Garçon

Exemple : Voiture



Définitions

- Notion d’Objet:
  - Une abstraction du monde réel c.-à-d. des données informatiques regroupant des caractéristiques du monde réel
- Caractérisation d’un objet
  - Identité: permet de le distinguer des autres objets,
  - Attributs (état): données caractérisant l'objet,
  - Méthodes (comportement): actions que l'objet est à même de réaliser.

Définitions

- Notion de Classe:
  - Une classe est une description abstraite des propriétés et comportements communs à un ensemble d'objets.
  - elle définit leur structure, leur comportement et leurs relations à l'aide des propriétés suivantes:
    - **Classe=attributs + méthodes + instanciation**

Une classe dans un programme

- une classe est une collection de données et des méthodes sur ces données.
- Une classe est une **encapsulation** des données et du code.
- La notion de l'encapsulation est la création des classes
  - donne une organisation naturelle des fonctions (méthodes)
  - permet d'effectuer la modification locale du code

Encapsulation

- Visibilité des attributs:
- Définissent les droits d'accès aux données (pour la classe elle-même, d'une classe héritière, ou bien d'une classe quelconque).

Concepts de POO

- **Modularité** : décomposition d'un système en modules cohérents et faiblement couplés
- **Abstraction** des données : ignorer les détails pour ne conserver qu'une notion générale.
- **Encapsulation** des données : on n'agit pas directement sur les données d'un objet. On interagit par l'intermédiaire de ses méthodes. Cela facilite la maintenance et la réutilisation.
- **Héritage** : permet de définir une nouvelle classe à partir d'une classe existante à laquelle on ajoute de nouvelles données et méthodes.
- **Polymorphisme** : donner plusieurs formes à une même classe d'objets en fonction des données.

Programmation C++

Spécificités du C++

Présentation du C++

- Le C++ est un langage **multiparadigme.** Il supporte essentiellement les paradigmes suivants :
  - **Programmation procédurale :** il reprend essentiellement les concepts du langage C, notamment la notion de fonction;
  - **Programmation structurée** : il reprend la notion **struct** du langage C. Cette notion est considérée en C++ aussi comme des classes dont l'accès par défaut est public;
  - **Programmation orientée-objet :** il implémente la notion de classe, d'encapsulation, d'héritage, d'abstraction, de polymorphisme;
  - **Programmation générique** : il implémente les fonctions et classes génériques.

C++ versus C

Par rapport au **langage C, le C++ apporte :**

- les concepts **orientés objet ;**
- les **références ;**
- les **valeurs par défaut des paramètres de fonctions;**
- la **surcharge** de fonctions**;**
- la **surcharge** des opérateurs;
- les **templates** de classes et de fonctions **;**
- les **constantes typées ;**
- La gestion des **exceptions**.

Opérateurs . et ::

- On accède aux données et fonctions-membres d'un objet grâce à la notation pointée :
- si **mon_objet** est une instance de **Ma_classe**, on écrit **mon_objet.donnee** (à condition que **donnee** figure dans la déclaration de **Ma_classe**, et que l'accès en soit possible).
- D'autre part, dans la définition d'une **fonction-membre**, on doit ajouter ***<nom de la classe>::*** *devant* le nom de la fonction.
- Par exemple, la définition d'une fonction-membre **truc()** de la classe **Ma_classe** aura la forme suivante :

  <type> Ma classe::truc(<déclaration de paramètres formels>)

  <instruction-bloc>
- L'appel se fait avec la notation pointée, par exemple : **mon_objet.truc()** ;

Chaînes de caractères

Le type (classe) **std::string** fournit les opérations courantes sur les chaînes de façon bien plus simple que les « chaînes » en C.

- Déclaration et initialisation

```
std::string s1;
std::string s2a("valinit"), s2b="valinit";
std::string s3a(s1), s3b=s1;
```

- Accès à un caractère s[i]
- Lecture au clavier std::cin >> s;
- Affichage std::cout << s;
- Longueur d’une chaîne s.length()

Tableaux C++ std::array

```
#include <iostream>
#include <array>
int max(std::array<int,5> t) {
    int m(t[0]);
    for (std::size_t i=1; i<5; ++i)
        if (t[i]>m) m=t[i];
    return m;}

int main() {
    std::array<int,5> tab {3,7,1,2,6};
    std::cout << max(tab) << "\n";
    std::array<int,5> tab2(tab);
    tab2[2]=12;         tab=tab2;
    return 0;}
```

Allocation dynamique en C++

Allocation simple

- int * i = new int;
- int * i = new int(3); Fixer la valeur initiale

Allocation d’une zone

- int * t = new int[taille];

Libération

- delete i;

Références en C++

- Une **référence** est une variable synonyme (qui porte le même nom) qu'une autre variable.
- Syntaxe (Déclaration d'une référence)
  - type & nomvar(var_référencée);

La variable référencée doit obligatoirement être précisée au moment de la *déclaration.*

- Exemple

```
int i;
int & j(i);
i=5;
std::cout << j; // 5
j=3;
std::cout << i; // 3
```

Références en C++

- Les **références** sont principalement utilisées pour le passage par référence d'un argument à une fonction. Dans ce cas, on ne doit pas préciser la variable référencée.
- Exemple

```
void f(int & a, int & b) {
a=2; b=3; }
...
int c,d;
f(c,d);
std::cout << c << " ␣ " << d; // 2 3
```

- Simplifie dans la plupart des cas le « passage par pointeur » de C.

Objets constants

Les règles suivantes s’appliquent aux objets constants :

- On déclare un objet constant avec le modificateur **const**
- On ne peut appliquer que des méthodes constantes sur un objet constant
- Un objet passé en paramètre sous forme de référence constante est considéré comme constant
- *Remarque : une méthode constante est tout simplement une méthode qui ne modifie aucun des attributs de l’objet. Il est conseillé de qualifier* ***const*** *toute fonction qui peut l’être.*

Méthodes constantes

- On peut surcharger une fonction membre non constante par une fonction membre constante :

```
class Point {
private:        int x, y;
public:
    int X() const { return x; }
    int Y() const { return y; }
    int& X() { return x; }
    int& Y() { return y; }
    void afficher() { cout << X() << "," << Y() << endl ; }
};
Point p1(1, 0);
p1.afficher(); // donne : 1,0
int r;   r = p1.X();      p1.Y() = r;
p1.afficher(); // donne : 1,1
```

Attributs constants

- Une donnée membre d'une classe peut être qualifiée **const**. Il est alors obligatoire de l'initialiser lors de la construction d'un objet, et sa valeur ne pourra par la suite plus être modifiée.

```
class Point {
private:
    const char lettre;
    int x, y;
public:
    Point(char lettre, int x, int y):lettre(lettre), x(x), y(y)
    ...
    void afficher() { cout << lettre << ":" << X() << "," << Y() << endl ; }
};

Point p1('A', 1, 0);
p1.afficher(); // donne : A:1,0
```

Passage par valeur / référence

- Passer par valeur une variable provoque la copie de cette variable.
- int maximum(std::array<int, 1000000> t);
- Cette copie est gênante dans le cas de structures de données de grande taille : coût de la copie.
- **Solution**. Passer le paramètre par référence.
  - int maximum(std::array<int, 1000000> & t);
- **Problème**. La fonction peut modifier le paramètre.
  - Bug difficile à trouver.
  - La signature de la fonction doit être une documentation.

Passage par valeur / référence

- Un passage par référence constante évite de faire une copie, mais empêche de modifier l’objet passé.

```
int maximum(std::array<int, 1000000> const & t);
```

- Le passage par référence constante doit être préféré à un passage par valeur pour les types composés (struct, conteneurs, classes).

C++ : Un programme minimal

Input / output streams

Include a header file

End line

```
#include <iostream>

int main() {

    std::cout << "Hello World!" << std::endl;

    return 0;

}
```

Standard output stream

string

Overloaded operator

C++ : Un peu de syntaxe

Saisie et affichage des données (communication avec les E/S):

- Saisir une variable au clavier: **cin**
  - **syntaxe : cin>>exp1>>exp2>>…>> expn;**
- Afficher une variable ou une phrase à l'écran: **cout**
  - **syntaxe : cout<<exp1<<exp2<<…<<expn;**

C++ : Un peu de syntaxe

```
#include <iostream>
int main(){
 std:: cout << "BONJOUR" << std::endl;
 return 0;}
```

- **Plus besoin d'utiliser std::**

```
#include <iostream>
using namespace std;
int main(){
 cout << "BONJOUR" << endl;
 return 0;}
```

Exercices : Concepts de base

- Sachant que la fonction **rand()** te retourne un nombre aléatoire comprit entre 0 et RAND_MAX en fonction de la valeur de **srand** et du temps actuel:
  - srand (time(NULL));
  - randVal = rand() % 100 + 1;
- Ecrire un programme qui demande à l'utilisateur de taper un nombre pour trouver la valeur secrète. Si le nombre entré est supérieur à notre valeur le programme lui demande d’entrer un nombre plus petit et l’inverse pour un nombre inférieur à notre valeur.

Programmation C++

- Classes et objets
- Fonction membre
- Constructeur de recopie

Définir une classe en C++

```
class NomClasse
{
    typeAttribut1 nomAttribut1;
    typeAttribut2 nomAttribut2;
    typeAttribut3 nomAttribut3;
    typeAttribut4 nomAttribut4;
    …
};
```

```
class CDate
{
    int Jour;
    int Mois;
    int Annee;
};
```

Objet en C++

Comment créer un objet priseBastille de type CDate ?

⇒ La classe CDate est un type comme un autre (int, char…)

```
{
        CDate priseBastille;
}
```

Initialiser un Objet

Pour l'instant, notre objet priseBastille n'est pas encore le 14 Juillet 1789.

⇒ Spécifier chaque attribut à la main

```
{
        priseBastille.Jour = 14;
        priseBastille.Mois = 07;
        priseBastille.Annee = 1789;
}
```

Initialiser un Objet

Autre exemple : la date armistice que l'on souhaite initialiser au 11 Novembre 1918

```
{
        CDate armistice;
        armistice.Jour=11;
        armistice.Mois=11;
        armistice.Annee=1918;
}
```

Comportement

Spécifier des comportements génériques communs à tous les objets de notre classe.

Par exemple :

- s'initialiser
- s'afficher
- vérifier si c'est contemporain

Initialiser

```
class CDate
{
    int Jour ;
    int Mois ;
    int Annee ;
```

```
void InitDate(int jour,
                int mois,
                int annee)
    {
        Jour = jour;
        Mois = mois;
        Annee = annee;
    }
};
```

Afficher

```
class CDate
{   int Jour ;
    int Mois ;
    int Annee ;

    void InitDate(int jour,
                  int mois,
                  int annee)
    {…}
```

```
    void Affiche()
    {
        cout << Jour <<  "/"
            << Mois << "/"
            <<Annee ;
    }
};
```

Contemporain

```
class CDate
{   int Jour;
    int Mois;
    int Annee;

    void InitDate(int jour,
                  int mois,
                  int annee)
    {…}
    void Affiche()
    {…}
```

```
    bool Contemporain()
    {
        if( Annee >= 2000 )
            return true;
        return false;
    }
};
```

Exemple

```
{
   CDate priseBastille; priseBastille.InitDate(14,07,1789);
   CDate armistice;
   armistice.InitDate(11,11,1918);

   cout <<"La date ";   priseBastille.Affiche();
    cout <<" est elle contemporaine ? "
          << priseBastille.Contemporaine()<<endl;

   cout <<"La date "; armistice.Affiche();
    cout <<" est elle contemporaine ? "
          << armistice.Contemporaine()<<endl;
}
```

Résultat

```
{
  CDate priseBastille;
  CDate armistice;

  priseBastille.InitDate(14,07,1789);
  cout <<"La date ";
  priseBastille.Affiche();
  cout <<" est elle contemporaine ? "
    <<priseBastille.Contemporaine()<<endl;

  armistice.InitDate(11,11,1918);
  cout <<"La date ";
  armistice.Affiche();
  cout <<" est elle contemporaine ? "
        <<armistice.Contemporaine()<<endl;
}
```

```
La date 14/07/1789 est elle
      contemporaine ? no

La date 11/11/1918 est elle
      contemporaine ? no
```

Initialiser un Objet – 2ème méthode

Il existe une autre méthode pour initialiser les attributs d'un objet :

$\Rightarrow$ utiliser les constructeurs.

Un constructeur est une "procédure" appelée <u>automatiquement</u> qui permet de spécifier chacun des attributs de l'objet en fonction de ses paramètres.

Un constructeur ne possède aucun type de retour, même pas void.

Constructeur par défaut

```
class CDate
{
    int Jour ;
    int Mois ;
    int Annee ;
    CDate()
    {/* constructeur par défaut qui initialise
          l'objet date au 01 Janvier 2005 */
          Jour = 1; Mois = 1; Annee = 2005;
    }
};
```

Créer mon objet

```
{
    CDate uneDate;

    cout << "La date ";
    uneDate.Affiche();
    cout<< " est elle contemporaine ? "
          << uneDate.Contemporaine();
}
```

Créer mon objet (suite)

Le programme va appeler automatiquement le constructeur (aucun paramètre).

L'objet uneDate sera donc initialisé avec

le 01 Janvier 2005

il appelle ensuite la fonction Affiche() pour afficher la date puis la fonction Contemporaine() pour savoir si elle est contemporaine de notre siècle.

On obtient :

La date 01/01/2005 est elle contemporaine ? yes

Autres Constructeurs Possibles

```
CDate (int jour, int mois, int annee)
{
    Jour = jour;
    Mois = mois;
    Annee = annee;
}

CDate (int jour, int mois)
{
    Jour = jour;
    Mois = mois;
    Annee = 2005;
}
```

Usage

```
{
    CDate uneDate;
    CDate priseBastille (14,07,1789);
    CDate dateCetteAnnee(7,03);
    uneDate.Affiche(); // 01/01/2005
    priseBastille.Affiche(); // 14/07/1789
    dateCetteAnnee.Affiche(); // 7/3/2005
}
```

Écriture d'un Constructeur

```
class MaClass
{
    type1 Attribut1;
  …
    MaClass(typeP1 parametre1, typeP2 parametre2, …)
    {
            …
    }
};
```

Sécurité

Comment empêcher un programmeur de définir la date 38/14/-362 ?

```
{
  CDate dateImpossible;
  dateImpossible.Jour=38;
  dateImpossible.Mois=14;
  dateImpossible.Annee=-362;
  dateImpossible.Affiche();
}
```

mots clés

Le C++ met à votre disposition 3 mots clés :

- public:
- private:
- protected:

public:

- Tous les attributs ou fonctions situés sous le mot clé public: sont accessibles en dehors de l’objet ou depuis n'importe quelles fonctions de la classe.

<u>Remarque</u> : Les fonctions précédentes ne fonctionnent qu'en ajoutant le mot clé public:

Exemple de public:

```
class CDate
{
  public:
    int Jour;
    int Mois;
    int Annee;

    void Affiche()
    {
       cout << Jour <<  "/" << Mois
    << "/"<< Annee;
    }
};
```

```
{
  CDate uneDate;


  uneDate.Jour=06;
  uneDate.Mois=10;
  uneDate.Annee=1289;


  uneDate.Affiche();
// 06/10/1289
}
```

private:

- Tous les attributs ou fonctions situés sous le mot clé private: ne sont accessibles que depuis les fonctions de l’objet.
- C'est le comportement par défaut si aucun mot clé n'est spécifié.

private:

```
class MaClasse
{
  public:
     int NbreAccessible;
  private:
     int NbreProtege;

     void Affiche() {
       cout<<NbreProtege<<end;
     }
  public:
     void LanceAffiche() {
       Affiche();
     }
};
```

```
{
  MaClasse objet;

  objet.NbreAccessible=3; // OK

  objet.NbreProtege=7;
// Impossible

  objet.Affiche(); // Impossible

  objet.LanceAffiche(); // OK
}
```

protected:

- Utile en ce qui concerne l'héritage. Nous verrons cela plus tard.
- Pour l'instant, protected: est équivalent à private:

Sécurité

Problème : Comment empêcher l'accès à certains attributs tout en donnant la possibilité de les modifier/consulter ?

$\Rightarrow$ Solution : Définir des accesseurs

Accesseur en lecture

Un accesseur en lecture est une fonction de la classe permettant de récupérer le contenu d'un de ses attributs.

```
int GetMois()
{
    return Mois;
}
```

Accesseur en écriture

Un accesseur en écriture est une procédure de la classe permettant de modifier le contenu d'un de ses attributs.

```
void SetMois(int mois)
{
    Mois = mois;
}
```

Protéger les attributs de la classe CDate

```
class CDate
{
    private:
        int Jour;
        int Mois;
        int Année;
    public:
        int GetMois()
        {
          return Mois;
        }
```

```
  void SetMois(int mois)
  {
     if( mois < 1 || mois >= 12 )
            mois=1;
     Mois=mois;
  }

  int GetJour(){ … }
  void SetJour(int jour ) { … }

  int GetAnnee() { … }
  void SetAnnee(int annee){ … }
};
```

Reprenons notre exemple

Essayons de définir la date 38/14/-362 ?

```
{
  CDate dateImpossible;
  dateImpossible.SetJour(38); // Jour = 1
  dateImpossible.SetMois(14); // Mois = 1
  dateImpossible.SetAnnee(-362); // Annee = 2005
  dateImpossible.Affiche(); // 1/1/2005
}
```

- *En C++, la programmation d'une classe se fait en trois phases :*
  - *déclaration,*
  - *définition,*
  - *utilisation*

Déclaration

- c’est la partie interface de la classe. Elle se fait dans un fichier dont le nom se termine par .h Ce fichier se présente de la façon suivante :

class Maclasse*{*

public:

//déclarations des données et fonctions-membres publiques

private:

//déclarations des données et fonctions-membres privées

};

Définition

- c’est la partie implémentation de la classe. Elle se fait dans un fichier dont le nom se termine par .cpp
- Ce fichier contient les définitions des fonctions-membres de la classe, c’est-`a-dire le code complet de chaque fonction.

Utilisation

- elle se fait dans un fichier dont le nom se termine par .cpp

Structure d’un programme en C++

- Nos programmes seront généralement composés d’un nombre impair de fichiers :
- pour chaque classe :
  - un fichier .h contenant sa déclaration,
  - un fichier .cpp contenant sa définition,
  - un fichier .cpp contenant le traitement principal.
- Ce dernier fichier contient la fonction main, et c’est par cette fonction que commence l’exécution du programme.

Exemple

- Le programme qui simule le fonctionnement d’un parcmètre se compose de trois fichiers :
  - parcmetr.h qui contient la déclaration de la classe Parcmetre,
  - parcmetr.cpp qui contient la définition de la classe Parcmetre,
  - simul.cpp qui contient l’utilisation de la classe Parcmetre.

// ---------- parcmetr.h ----------

- // ce fichier contient la déclaration de la classe Parcmetre

```
class Parcmetre{
public:
    Parcmetre(); // constructeur de la classe
    void Affiche(); // affichage du temps de stationnement
    void PrendsPiece(float valeur); // introduction d'une pièce
private:
    int heures, // chiffre des heures...
    minutes; // et des minutes
```

- *};*

```
// ------------------------ parcmetr.cpp ---------------------
// ce fichier contient la définition de la classe
   Parcmetre
#include <iostream.h> // pour les entrées-sorties
#include "parcmetr.h" // déclaration de la classe
   Parcmetre
Parcmetre::Parcmetre() // initialisation d'un
   nouveau parcmètre
{ heures = minutes = 0;}
```

```
void Parcmetre::Affiche() // affichage du temps de
    stationnement restant
// et du mode d'emploi du parcmètre
{cout << "\n\n\tTEMPS DE STATIONNEMENT :";
cout << heures << " heures " << minutes << " minutes";
cout << "\n\n\nMode d'emploi du parcmètre :";
cout << "\n\tPour mettre une pièce de 10 centimes : tapez
    A";
cout << "\n\tPour mettre une pièce de 20 centimes : tapez
    B";
cout << "\n\tPour mettre une pièce de 50 centimes : tapez
    C";
cout << "\n\tPour mettre une pièce de 1 euro : tapez D";
cout << "\n\tPour quitter le programme : tapez Q";
}
```

```
void Parcmetre::PrendsPiece(float valeur) // introduction
    d'une pièce
{
minutes += valeur * 10; // 1 euro = 100 minutes de
    stationnement
while (minutes >= 60){
    heures += 1;
    minutes -= 60;}
if (heures >= 3) // on ne peut dépasser 3 heures
{
heures = 3;
minutes = 0;
}
}
```

```
// ------------------------ simul.cpp --------------------
// ce fichier contient l'utilisation de la classe Parcmetre
#include <iostream.h> // pour les entrées-sorties
#include "parcmetr.h" // pour la déclaration de la classe Parcmetre
void main() // traitement principal{
    Parcmetre p; // déclaration d'un parcmètre p
    char choix = 'X';
    while (choix != 'Q') // boucle principale d'événements{
        p.Affiche();
        cout << "\nchoix ? --> ";
        cin >> choix; // lecture d'une lettre
        switch (choix) // action correspondante{
            case 'A' : p.PrendsPiece(1); break;
            case 'B' : p.PrendsPiece(2); break;
            case 'C' : p.PrendsPiece(5); break;
            case 'D' : p.PrendsPiece(10);}}}
```

Allocation dynamique en C++

- le C++ fournit d’autres moyens pour allouer et restituer la mémoire. Pour cela, il dispose d’opérateurs spécifiques : new, delete, new[] et delete[].
- La syntaxe de ces opérateurs est respectivement la suivante :
  - new type
  - delete pointeur
  - new type[taille]
  - delete[] pointeur

Pointeur this

- le pointeur sur l’objet est accessible à l’intérieur de la fonction membre. Il porte le nom « **this** ». Par conséquent, ***this** représente l’objet lui-même.

Données et fonctions membres **statiques**

- Une classe peut contenir des données membres statiques. Ces données sont déclarées avec le mot-clé **static**.
- Elles appartiennent à la classe, et non pas aux objets de cette classe.
- on devra utiliser l'opérateur de résolution de portée

(: :).

- class test {
  - static int i; // Déclaration dans la classe.
  - ... };
- int test::i=3; // Initialisation en dehors de la classe

Appel de fonction membre statique

```
class Entier {
static int i;
public:
   static int get_value(void);
};
int Entier::i=3;
int Entier::get_value(void) { return i; }
int main(void) { // Appelle la fonction statique get_value :
    int resultat=Entier::get_value();
    return 0; }
```

Destructeur

- Le nom du destructeur est toujours précédé du signe tilde ('~').
- Le destructeur est appelé automatiquement lors de sa destruction.
- Cette destruction a lieu lors de la sortie du bloc de portée courante pour les objets de classe s'il n'est pas alloué dynamiquement